

Digital Circuits: A Review

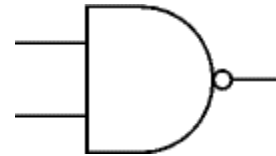
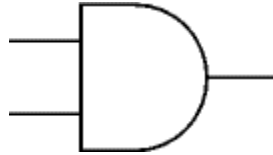
Lesson 1

Gates - Introduction

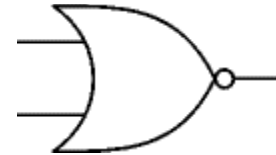
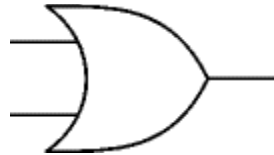
- Logic gates or simply gates are key components that are part of a digital circuit. They are the basic building blocks of a computer.
- They help to define a logical sequence of actions that take place under given circumstances.
- Gates are basic circuits that have **at least one** (and usually more) *input* and **exactly** one output.
- Input and output values are the logical values *true* and *false* (1 or 0, high or low, on or off)
- Usually three basic kinds of gates are considered and are identified by their function
→ **AND**-gates, **OR**-gates, and **NOT**-gates (or *inverters*).

Basic Gates

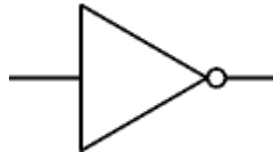
■ AND & NAND



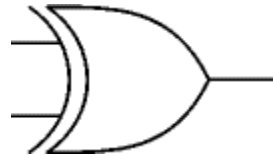
■ OR & NOR



■ NOT

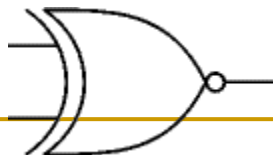


■ XOR -



**$Q = (A \text{ AND NOT } B)$
 $\text{OR } (B \text{ AND NOT } A)$**

■ EX-NOR

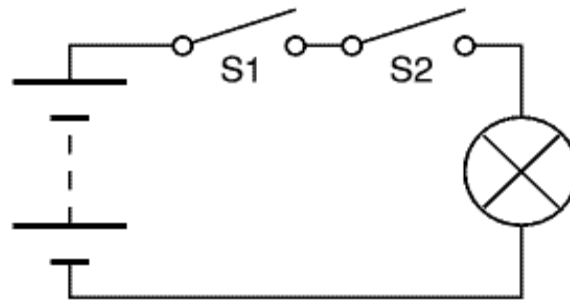


**$Q = (A \text{ AND } B)$
 $\text{OR } (\text{NOT } A \text{ AND NOT } B)$**

Simple circuit with two switches connected in series

- **Note** that logic gates are not always required because **simple logic functions** can be performed with switches or diodes:

- **AND**



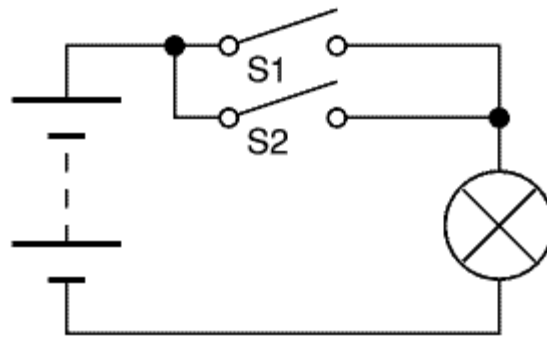
simple circuit with two switches connected in series to control a lamp

Switch S1 **AND** Switch S2 must be closed to light the lamp

Simple circuit with two switches connected in parallel

- If several on-off switches are connected in parallel only one needs to be closed (on) to complete the circuit.

■ OR



simple circuit with two switches connected in parallel to control a lamp.

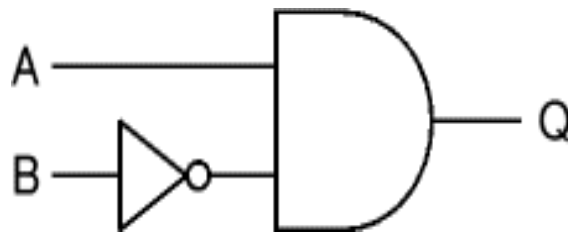
Switch S1 OR Switch S2 (or both of them) must be closed to light the lamp.

Combined Gates

- Sometimes, it is practical to **combine functions of the basic gates into more complex gates**, for instance in order to save space in circuit diagrams.
- NAND gate, NOR gate, Exclusive-OR gate
- Gates can also be combined to **substitute one type of gate for another**.

Combination of Logic Gates

- To produce an output Q which is **true only when input A is true and input B is false**, we can combine a NOT gate and an AND gate like this:

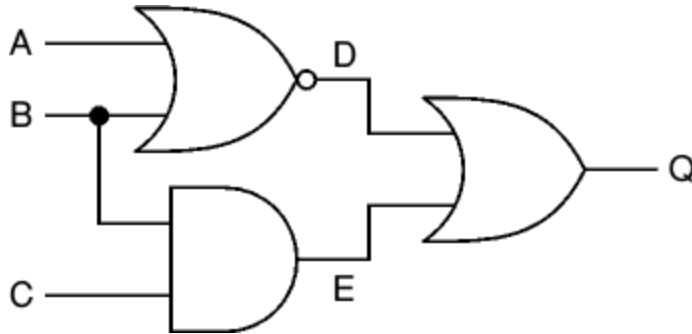


$$Q = A \text{ AND NOT } B$$

Input A	Input B	Output Q
0	0	0
0	1	0
1	0	1
1	1	0

Working out the function of a combination of gates

- Truth tables can be used to work out the function of a combination of gates.



$D = \text{NOT } (A \text{ OR } B)$

$E = B \text{ AND } C$

$Q = D \text{ OR } E = (\text{NOT } (A \text{ OR } B)) \text{ OR } (B \text{ AND } C)$

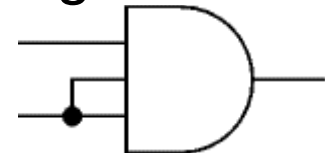
Inputs				Outputs	
A	B	C	D	E	Q
0	0	0	1	0	1
0	0	1	1	0	1
0	1	0	0	0	0
0	1	1	0	1	1
1	0	0	0	0	0
1	0	1	0	0	0
1	1	0	0	0	0
1	1	1	0	1	1

Substituting one type of gate for another

- Logic gates are available on ICs which **usually contain several gates of the same type**, for example four 2-input NAND gates or three 3-input NAND gates
- To avoid using too many ICs you can **reduce the number of gate inputs or substitute one type of gate for another.**

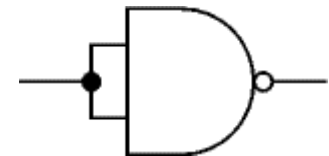
(a) **Reducing the number of inputs to a gate**

- by connecting two (or more) inputs together. The diagram shows a 3-input AND gate operating as a 2-input AND gate.



(b) **Making a NOT gate from a NAND or NOR gate**

- Reducing a NAND or NOR gate to just one input creates a NOT gate. The diagram shows this for a 2-input NAND gate.



- **Any gate** can be built from NAND or NOR gates hence a circuit can be built from just one type of gate, either NAND or NOR

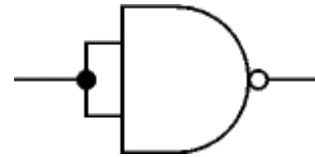
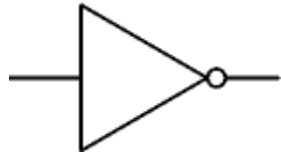
- **To change the type of gate**, such as changing OR to AND, you must do three things:
 - ❑ Invert (NOT) each input.
 - ❑ Change the gate type (OR to AND, or AND to OR)
 - ❑ Invert (NOT) the output.
- e.g. OR gate can be built from NOTed inputs fed into a NAND (AND + NOT) gate.

Doing it all with only one kind of gate

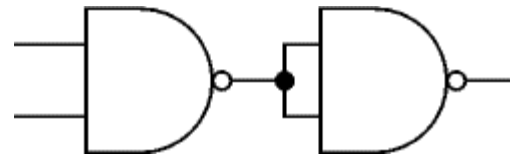
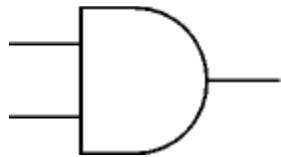
- It is possible to build any kind of gate using only *nand*-gates.
- An *inverter* is just a **NAND**-gate with only one input.
- An **AND**-gate can be built as a **NAND** with an inverter on its output.
- An **OR**-gate can be build with a **NAND**-gate with an *inverter* on each input.
- Similarly, all gates can be realized from only **NOR**-gates. Again an *inverter* is just a **NOR**-gate with only one input. An **OR**-gate is a **NOR**-gate with an *inverter* on its output, and an **AND**-gate is just a **NOR**-gate with an *inverter* on each input.

NAND gate equivalents of NOT, AND, OR and NOR gates

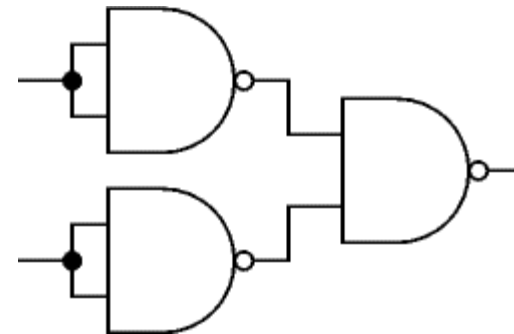
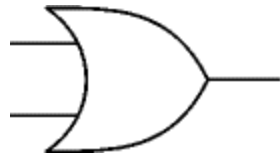
■ NOT



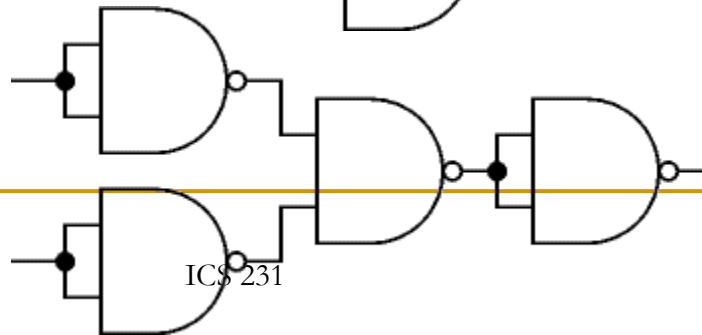
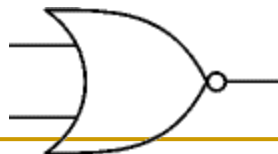
■ AND



■ OR



■ NOR



Truth Tables

- To specify the exact way in which a combinational circuit works, we might use different methods:
 - Logical expressions
 - Truth tables
- A truth table is a complete enumeration of **all possible combinations of input values**, with each being associated with **its output value**.
- Truth tables are used to:
 - Describe existing circuits
 - Specify new circuits to be built

Combinational Circuits

- It is an interconnected set of gates whose output at any time is a function only of the input at that time.
- In general such a circuit has m **inputs** and n **outputs**.
- When we build a combinational circuit from some kind of specification, we always try to **satisfy some criteria** such as:
 - Minimize number of gates (reduce cost)
 - Lowest power consumption
 - Shortest delay possible

Types of Combinational Circuits

■ Multiplexer

- A multiplexer is a combinational circuit that is given **a certain number (usually a power of two) data inputs**, let us say 2^n , and **n address inputs** used as a **binary number to select one of the data inputs**.
- The multiplexer has **a single output**, which has the **same value as the selected data input**.
- <http://dept-info.labri.fr/~strandh/Teaching/AMP/Common/Strandh-Tutorial/multiplexer.html>

■ Demultiplexer

- it takes a **single data input** and **n address inputs**. It has **2^n outputs**. The address input determines **which data output is going to have the same value as the data input**. The other data outputs will have the value 0.
- <http://www.labri.fr/perso/strandh/Teaching/AMP/Common/Strandh-Tutorial/demultiplexer.html>

■ There are many others, like **decoders**, etc

Latches and Flip-flops

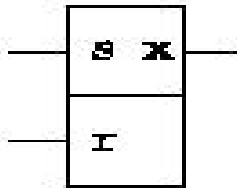
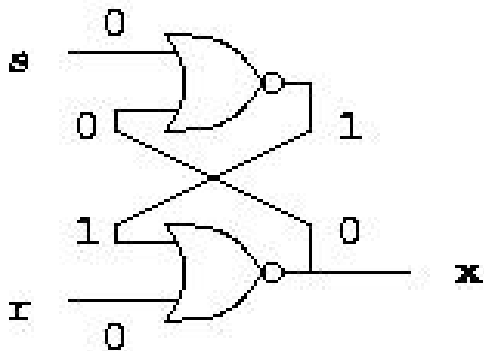
- Gates are the building blocks of **combinational circuits**
- Latches and flip-flops are the building blocks of ***sequential circuits***
- Both latches and flip-flops are circuit elements whose output depends not only on the current inputs, but also on previous inputs and outputs.
- The difference between a latch and a flip-flop is that a latch does not have a *clock signal*, whereas a flip-flop always does.

Latches and Flip-flops

- **Gates** had to be built directly from **transistors**, **latches** can be built **from gates**, and **flip-flops** can be built **from latches**.

Latches

- How can we make a circuit out of gates that is **not combinational**?
- Use ***feed-back***-, create ***loops*** in the circuit diagrams so that output values depend, indirectly, on themselves.
- Simple *set* and *reset* latch



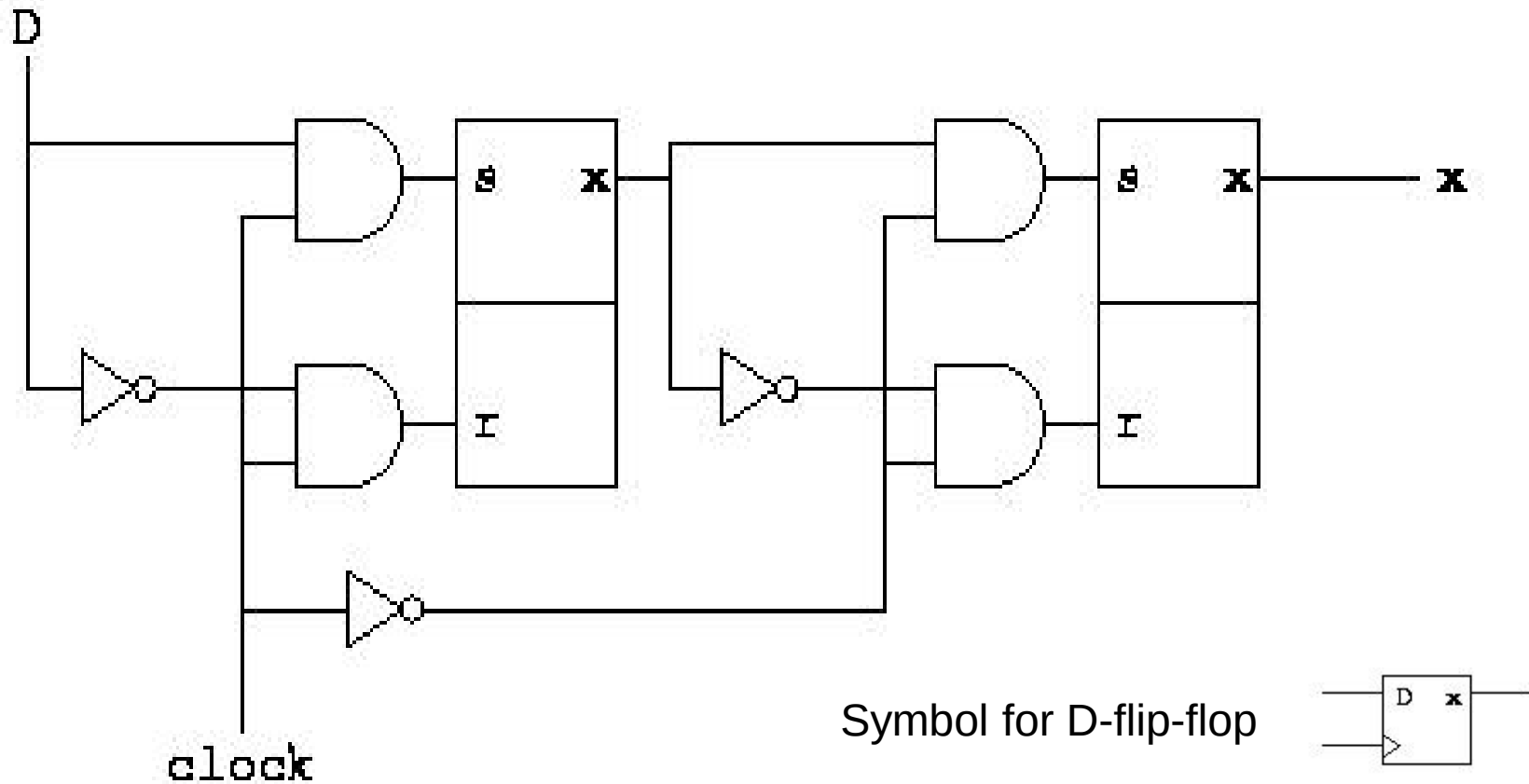
The actual value of output (x) depends on the *history* of input values.

Flip-flops

- Latches are *asynchronous*, which means that the **output changes soon after the input changes**.
- Most computers today, on the other hand, are *synchronous*, which means that the **outputs of all the sequential circuits change simultaneously to the rhythm of a global *clock signal***.
- A *flip-flop* is a synchronous version of the latch.
- <http://www.labri.fr/perso/strandh/Teaching/AMP/Common/Strandh-Tutorial/flip-flops.html>

Flip-flops -A master-slave D-flip-flop

Built from two SR-latches and some gates



Sequential Circuits

- We define a *synchronous sequential circuit*, or just ***sequential circuit*** as a circuit with ***m*** inputs, ***n*** outputs, and a **distinguished clock input**.
- The description of the circuit is made with the help of a ***state table*** which show ***transition from state to state***.

References

- Computer Organization and Architecture, William Stallings – Appendix A: Digital Logic
- Computer Organization and Design, David A. Patterson & John L. Hennessy – Appendix B: The Basics of Logic Design
- Computer Architecture and Logic Design, Thomas C. Bartee – Chapter 3 & 4.
- Wikipedia